Programming Homework: Genetic Algorithms

Problem 1: Genetic Algorithm Structure and Reproduction Operators

Requirements:

Implement a complete Genetic Algorithm (GA) framework from scratch in Python to optimize a binary bit string (OneMax problem) based on the mechanisms introduced in the lecture slides.

a) Hypothesis Initialization and Encoding:

- Randomly initialize an initial population P consisting of $p = 50$ hypotheses (binary bit strings of length $L = 20$).
- Define the fitness function $Fitness(h)$ as the total count of 1-bits in the bit string h.

b) Implement the Generation Creation Mechanism:

- **Selection:** Implement fitness-proportionate selection (Roulette Wheel Selection) with probability $Pr(h_i) = \frac{Fitness(h_i)}{\sum_{h \in P} Fitness(h)}$ to directly retain $(1 - r) \cdot p$ individuals.
- **Crossover:** Select $(r/2) \cdot p$ pairs of parents according to $Pr(h)$ and implement 3 crossover operators: Single-point, Two-point, and Uniform Crossover with a crossover rate of $r = 0.8$.
- **Mutation:** Apply mutation to $m = 5\%$ of the individuals added to the new population by randomly flipping 1 bit in their string.

c) Evaluation and Performance Comparison:

- Run the algorithm over 100 generations or stop when a hypothesis with $Fitness(h) = 20$ is found.
- Experiment with and compare the convergence speed of the 3 crossover methods (Single-point, Two-point, Uniform).
- Plot graphs showing average Fitness and maximum Fitness over generations for each method.

Hints:

- Use the `numpy` library to manage bit arrays and calculate selection probabilities.
- Use the `random` module to select crossover points and bit mutation positions.
- Use `matplotlib.pyplot` to visualize convergence progress over generations.

Problem 2: Rule Representation and GA Hyperparameter Optimization

Requirements:

Implement a rule-based classification system using bit strings and investigate the impact of hyperparameters on GA performance.

a) Rule Representation using Bit Strings (Hypothesis Representation):

- Build a class to encode an IF-THEN rule into a bit string following the document structure: $[A_1 \mid A_2 \mid A_3 \mid C]$, where A_1 has 3 possible values (3 bits), A_2 has 2 possible values (2 bits), A_3 has 2 possible values (2 bits), and Target Class $C \in \{0, 1\}$ (1 bit).
- Implement a function to decode bit strings into logical rules and a function to predict labels for data samples.
- Implement a squared fitness function: $Fitness(h) = (correct(h))^2$, where $correct(h)$ is the percentage of training samples correctly classified.

b) Investigate the Impact of Crossover Rate r and Mutation Rate m:

- Experiment with crossover rate values $r \in [0.2, 0.5, 0.8, 1.0]$.
- Experiment with mutation rates $m \in [0.01, 0.05, 0.1, 0.2]$.
- Evaluate how changing these values affects the accuracy of the discovered rule and the algorithm's convergence speed.

c) Variable-length Crossover Experiment:

- Extend the problem to allow individuals to contain a set of multiple concatenated rules (concatenated bit strings).
- Implement variable-length crossover operators between individuals with different numbers of rules.
- Analyze the changes in the average number of rules per population across generations.

Hints:

- Create a small synthetic dataset containing attributes A_1, A_2, A_3 to evaluate the fitness function.
- Use the `time` function to measure execution time when varying parameters r and m.
- Plot heatmaps or line charts to compare the obtained accuracy across different parameter pairs (r, m).